WEEK 5

INTRODUCTION TO DESIGN PATTERNS

Mr Randell Gaya

Module Originator:
Dr Tom Blanchard,
Dr Lee Clift



TODAY'S GOALS

Design patterns:

- Motivation.
- What they are.

Main ideas:

- Program to Interface, not Implementation.
- Favour Composition over Inheritance.
- Delegation.

What main design patterns exist.

HOW DO WE DESIGN?

We **do not want** to keep producing new designs from first principals.

When facing a **new problem** we often think:

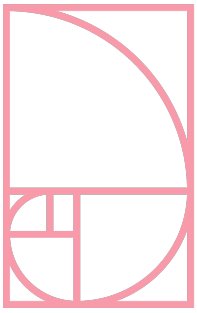
*"well, how did I (or others) solve a similar problem **before**?"*

We often repeat **patterns of design** that have worked in the past.

Design patterns can be applied to software design to achieve certain **goals** (with certain consequences).

- This is a common approach in many other fields.

DESIGN PATTERNS: GOALS



1. To support **reuse** of successful designs.
2. To facilitate **software evolution.**
3. In short, we want to **design for change.**

DESIGN PATTERNS: AN EXAMPLE

Iterators are an example of a design pattern:

- Design pattern = problem + solution in context.
- Iterators: solution for providing generic traversals.

Don't really care **what** I'm traversing.

How I am traversing the data structure could change.

```
public class Main{

  public static void main(String[] args){
    List<String> someStrings = new ArrayList<>();

    Iterator<String> it = someStrings.iterator();

    while(it.hasNext()) {
      System.out.println(it.next());
    }
  }
}
```

DESIGN PATTERNS

Design patterns capture software architectures and designs.

- Generic blueprints.
- Language and implementation independent.
- Not direct code reuse!
- Instead, solution/strategy reuse.
- Sometimes, interface reuse.

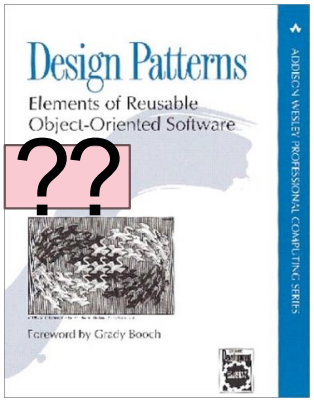
GANG OF FOUR (GOF)

The book that 'started it all'.

Authors referred to as the **Gang of Four**.

While designs are generic, the sample code
in the GOF book uses C++ and small talk, not Java.

Uses OMT not UML. OMT eventually evolved to become UML. OMT syntax varies a little but captures the same ideas as UML.



DESIGN PATTERNS

"Design patterns capture solutions that have developed and evolved over time. Hence they aren't the designs people tend to generate initially. They reflect untold redesign and recoding as developers have struggled for greater reuse and flexibility in their software."

Gang of Four

May be more work than an *ad hoc* solution…

COMPONENTS OF A PATTERN

What is a Design Pattern?

- **Basic idea:** Common solution to a reoccurring problem in design.

Each design pattern has/describes:

- **Name:** The name of the Pattern
- **Problem:** What the problem solves, the context in which it is solved, and a real world example
- **Solution:** The design/structure of the solution, and its implementation
- **Consequences:** The consequences on client code of the implementation
- **Variations:** Possible variations of the pattern
- **Known Uses:** Known uses of the pattern

UNDERLYING PRINCIPLES

Reduce implementation dependencies between elements of a software system.

Sub-goals:

- Program to an interface, not an implementation.
- Favour composition over inheritance.
- Use delegation.

UNDERLYING PRINCIPLES

PROGRAM TO INTERFACE, NOT IMPLEMENTATION PRINCIPLE



PROGRAM TO INTERFACE, NOT IMPLEMENTATION

Rely on abstract classes and interfaces to hide differences between subclasses from client code.

- Interface defines an object's use (protocol).
- Implementation defines particular policy.

PROGRAM TO INTERFACE, NOT IMPLEMENTATION: RATIONALE

1. Decouples clients (client code) from the implementations of the applications they use.
2. When clients manipulate an interface, they remain unaware of the specific object types being used.
3. Therefore: clients are less dependent on an implementation, so it can be easily changed later.

UNDERLYING PRINCIPLES

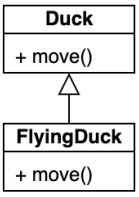
FAVOUR COMPOSITION OVER CLASS INHERITANCE PRINCIPLE



FAVOUR COMPOSITION OVER CLASS INHERITANCE

White box reuse:

- Inheritance.

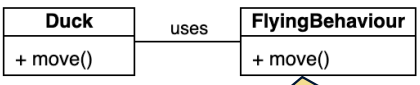


Inheritance is not bad, but it can be better to use composition of objects in some cases.

Inheritance is fixed at compile time

Black box reuse:

- Composition.



Instances of Duck can forward the calls on to FlyingBehaviour. This can simulate inheritance.

Composition can be changed at run-time.

We can reuse the FlyBehaviour if we need it elsewhere!

FAVOUR COMPOSITION OVER CLASS INHERITANCE: RATIONALE

White-box reuse results in implementation dependencies on the **parent class.**

- Reusing a subclass may require **rewriting** the parent.
- But inheritance is **easy to specify**.

Black-box reuse **often preferred**.

- Eliminates implementation dependencies, hides information, object relationships are non-static for better run-time flexibility.
- But adds **run-time overhead** (additional instance allocation, communication by dynamic dispatch).

UNDERLYING PRINCIPLES

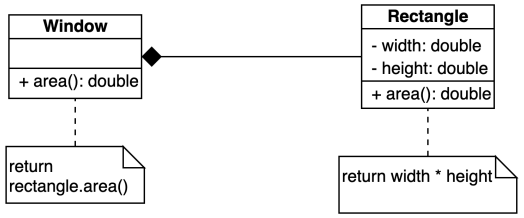
DELEGATION PRINCIPLE



DELEGATION

Forward messages (delegate) to different instances **at run-time**; a form of composition.

- Can pass a reference to the calling object to help simulate inheritance.



We could swap out the Rectangle for an Ellipse or some other shape at run-time for a funky shaped window.

The window will forward all necessary calls to the delegate.

DELEGATION: RATIONALE

With delegation the object relationships are dynamic.

- Composes or re-composes behaviour at run-time.

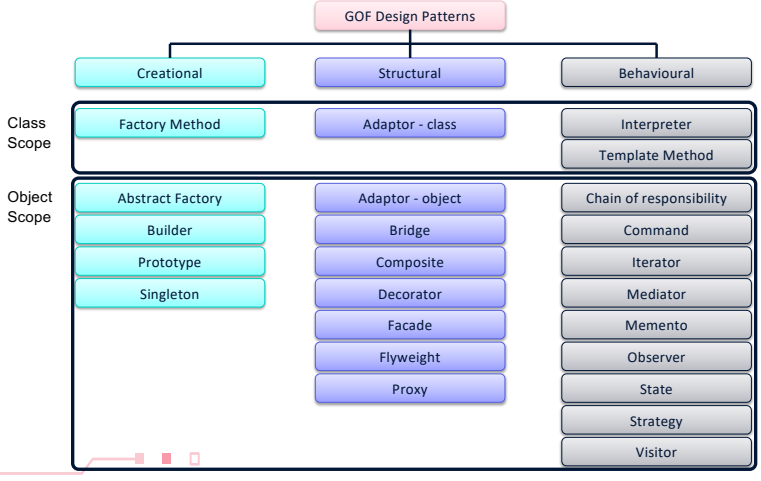
But:

- Sometimes code harder to read and understand.
- Efficiency (because of black-box reuse).

DESIGN PATTERNS TAXONOMY



DESIGN PATTERNS TAXONOMY



DESIGN PATTERNS TAXONOMY

Purpose based classification

Creational patterns.
- Concern the process of object **creation**.

Structural patterns.
- Deal with the **composition** of classes or objects (How to assemble objects/classes into larger structures).

Behavioural patterns.
- Characterize the ways in which classes or objects **interact** and distribute **responsibility**.

DESIGN PATTERNS TAXONOMY

Scope based classification.

Class scope.
- Relationship between classes & subclasses.
- Statically defined at run-time.

Object scope.
- Object relationships.
- Can be manipulated at runtime.

DESIGN PATTERNS TAXONOMY

Class Scope	Object Scope
Creational class defers object creation to sub-classes (*factory method*)	**Creational object** defers object creation to other objects (*abstract factory*)
Structural class inheritance to compose classes (*adapter*)	**Structural object** deals with object assembly (*adapter*)
Behavioural class uses inheritance to describe flow of control, algorithms (*template*)	**Behavioural object** group of objects working together to carry out a task (*iterator*)

SUMMARY

1. What are Design Patterns?
- What are the goals?

2. Principles motivating Design Patterns
- Programming to interfaces
- Composition over inheritance
- Delegation

3. Taxonomy of Design Patterns
- Scope vs Behavior

WEEK 5

THE NEED FOR A DESIGN PATTERN

Mr Randell Gaya

Module Originator:
Dr Tom Blanchard,
Dr Lee Clift



THE SIMUDUCK APP

Joe works for a company that makes a highly successful duck pond simulation game, ***SimUDuck***.

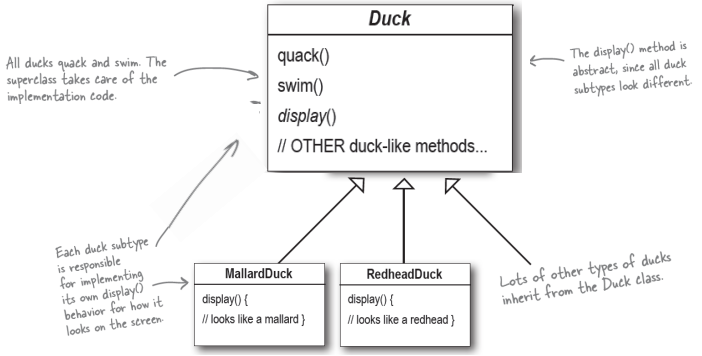
The game can show a large variety of duck swimming and making quacking sounds.

The initial designers of the system used standard OO techniques:
- They created one Duck superclass from which all other duck types inherit.

Seems pretty sensible, right?



ORIGINAL DESIGN



LIFE MOVES ON... DUCKS MUST FLY

In the last year, the company has been under increasing pressure from competitors.

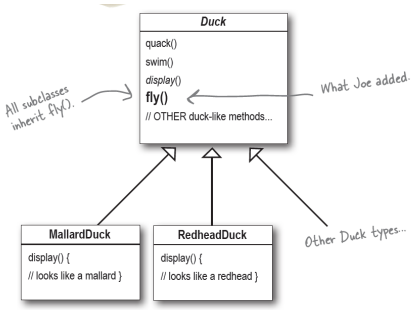
After a week of 'brainstorming sessions' over golf, the company executives think it's time for a **big innovation**.

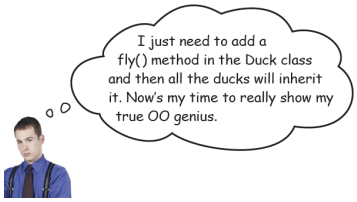
- They need something *really* impressive to show at the upcoming shareholders meeting in Maui *next week*.
- *"flying ducks is just what the simulator needs to blow away the other duck sim competitors."* – Some Executive

And of course Joe's manager told them it'll be no problem for Joe to just whip something up in a week. "After all," said Joe's boss, "he's an OO programmer... *how hard can it be?"*

JUST ADD A FLY() METHOD







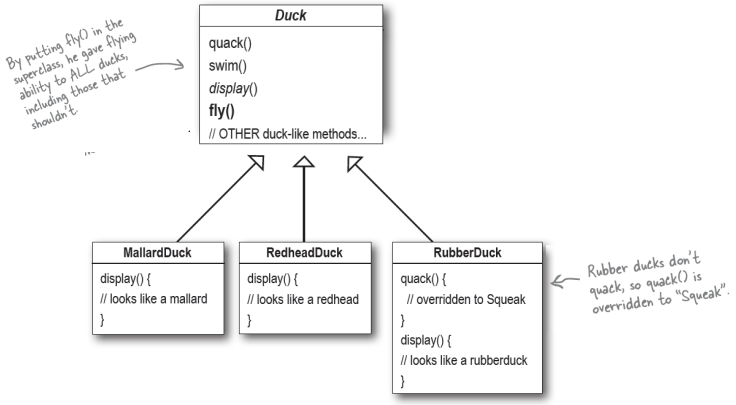
THE PROBLEM

In the demo, there were **rubber duckies** flying around the screen.

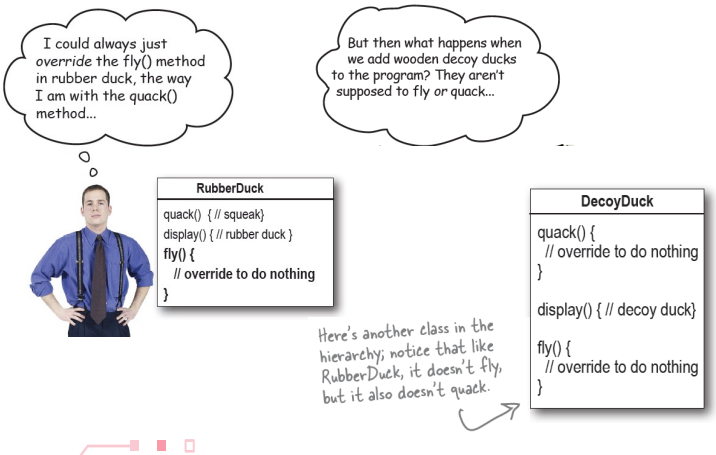
Joe failed to notice that not *all* subclasses of Duck should *fly*.

- When Joe added new behavior to the Duck superclass, he was also adding behavior that was *not* appropriate for some Duck subclasses.

What Joe thought was a great use of **inheritance** for the purpose of reuse hasn't turned out so well when it comes to **maintenance** and further development.



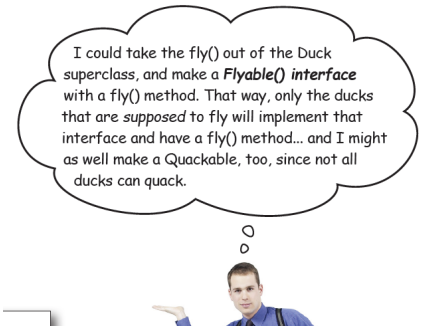
TRYING MORE INHERITANCE TRICKS



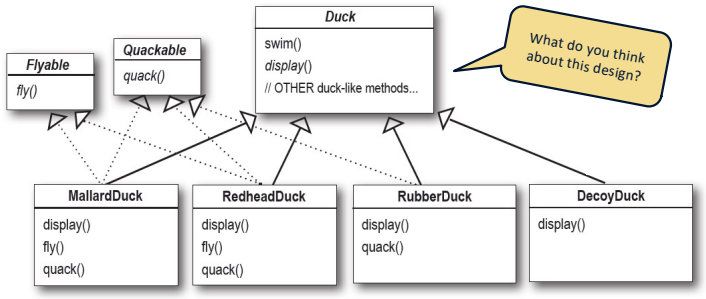
INHERITANCE IS NOT THE ANSWER

Joe knows the spec will keep changing and he'll be forced to look at and possibly override fly() and quack() for every new Duck subclass that's ever added to the program... *forever.*

How about an interface?



USE INTERFACES



What about duplicate code?
If you thought having to override a few methods was bad, how are you going to feel when you need to make a little change to the flying behavior... in **all** the flying Duck subclasses?! Imagine if there were 30 subclasses!

DUCK UTOPIA

Wouldn't it be wonderful if there were a way to build software so that when we need to change it, we could do so with the least possible impact on the existing code?

- We could spend *less time* reworking code and more making the program do cooler things...

The one constant thing in software development is **CHANGE**.

- Requirements.
- Business context.
- Tools.
- Customers.

THE PROBLEM

Inheritance has **not worked** out well:

- The Duck behaviour keeps changing across the subclasses.
- It's not appropriate for *all* subclasses to have those behaviours.

Flyable and Quackable **interface** sounded promising at first.

- Java interfaces have no implementation, so no code reuse.
- Modifying behaviour requires you to track down and change multiple different subclasses where that behaviour is defined.

ENCAPSULATION



Design Principle

Identify the aspects of your application that vary and separate them from what stays the same.

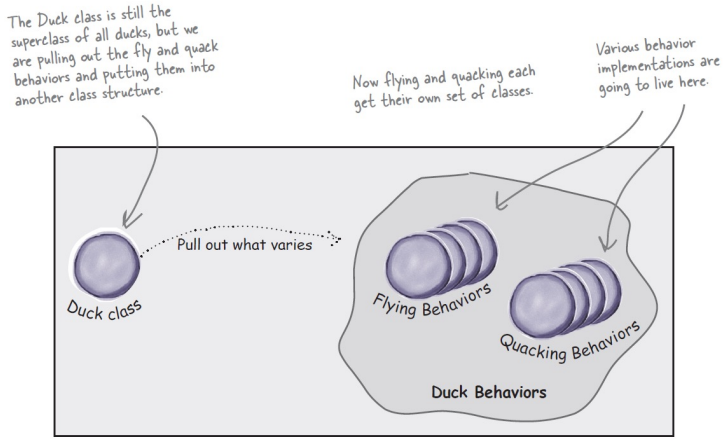
Take the parts that vary and encapsulate them, so that later you can alter or extend the parts that vary without affecting those that don't.

Fewer **unintended consequences** from code changes and more **flexibility** in your systems.

If you've got some aspect of your code that is **changing frequently**.

- With every new requirement.
- You've got a behaviour that needs to be pulled out and separated from all the code that doesn't change.

SEPARATING CODE THAT CHANGES



DESIGNING THE DUCK BEHAVIOUR

Now the Duck behaviours will live in a **separate class**.

- This class that implements a particular behaviour interface.

The Duck classes won't need to know any of the implementation details for their own behaviours.



Design Principle

Program to an interface, not an implementation.

PROGRAM TO AN INTERFACE, NOT BEHAVIOUR

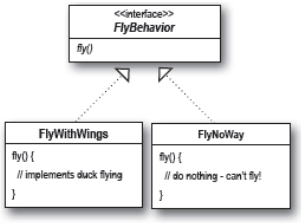
Use an **interface** to represent each behaviour.

- **FlyBehavior** and **QuackBehavior.**

Each implementation of a **behaviour** will implement one of those interfaces.

Duck classes won't implement the flying and quacking interfaces.

- Instead, we'll make a set of classes represent a behaviour (for example, "FlyWithWings"), and it's the *behaviour* class, rather than the Duck class, that will implement the behaviour interface.



CONTRAST TO EARLIER DESIGN

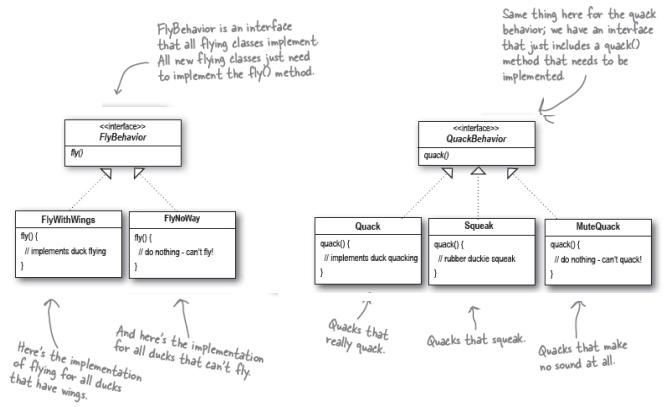
Previous design: a behaviour came either from:

- A **concrete implementation** in the superclass Duck. Or
- By providing a **specialised implementation** in the subclass itself.
- Relied on and locked into using an **implementation**. No room for changing the behaviour (other than writing more code).

New Design: the Duck subclasses will use a behaviour represented by an **interface** (FlyBehavior and QuackBehavior).

- The actual **implementation** of the behaviour (the specific behaviour coded in the class that implements the FlyBehavior or QuackBehavior) won't be locked into the Duck subclass.

IMPLEMENTING THE DUCK BEHAVIOURS



BENEFITS OF THIS DESIGN

Other types of objects can **reuse** our fly and quack behaviours because these behaviours are no longer hidden away in our Duck classes!

And we can **add new** behaviours without modifying any of our existing behaviour classes or touching any of the Duck classes that use flying behaviours.

So we get the benefit of **REUSE** without all the baggage that comes along with inheritance.

QUESTION...

It feels a little weird to have a class that's just a behaviour.

1. Aren't classes supposed to represent things?
2. Aren't classes supposed to have both state AND behaviour?

Answer:

- In an OO system, yes, classes represent things that generally have both state (instance variables) and methods, and in this case, the thing happens to be a behaviour.
- But even a behaviour can still have state and methods; a flying behaviour might have instance variables representing the attributes for the flying (wing beats per minute, max altitude, and speed, etc.) behaviour.

IMPROVING THE DUCK CLASS

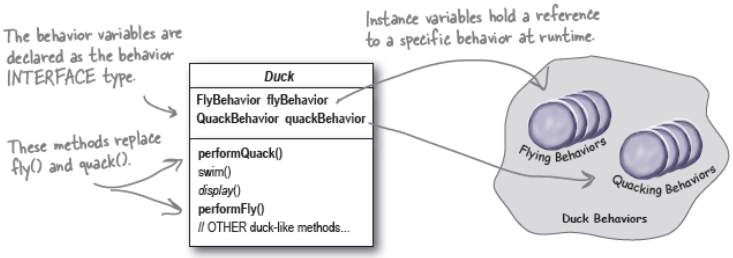
A Duck now **delegates** its flying and quacking behaviour, instead of using quacking and flying methods defined in the Duck class (or subclass).

First we'll add two instance variables to the Duck class called *flyBehavior* and *quackBehaviour*.

- Each duck object will set these variables to reference the **specific behaviour** type it would like at runtime (FlyWithWings, Squeak, etc.).
- We'll also **remove** the fly() and quack() methods from the Duck class because we've moved this behaviour out into the FlyBehaviour and QuackBehaviour classes.

IMPROVING THE DUCK CLASS

We'll **replace** fly() and quack() in the Duck class with two similar methods, called performFly() and performQuack()



IMPLEMENT PERFORMQUACK()

```
public abstract class Duck{

    protected QuackBehaviour quackBehaviour;

    public void performQuack(){
        quackBehaviour.quack();
    }
}
```

Rather than handling the quack behavior itself, the Duck object delegates that behavior to the object referenced by quackBehavior.

To perform the quack, a Duck just allows the object that is referenced by quackBehaviour to quack for it.

We don't care what kind of object it is, ***all we care about is that it knows how to quack()!***

INITIALISING THE BEHAVIOURS

Consider a Mallard Duck

```
public class MallardDuck extends Duck{

    public MallardDuck(){
        quackBehaviour = new Quack();
        flyBehaviour = new FlyWithWings();
    }

    public void display() {
        System.out.println("I'm a real Mallard Duck!");
    }
}
```

A MallardDuck uses the Quack class to handle its quack, so when performQuack() is called, the responsibility for the quack is delegated to the Quack object and we get a real quack.

And it uses FlyWithWings as its FlyBehavior type.

Remember, MallardDuck inherits the quackBehavior and flyBehavior instance variables from class Duck.

THE MALLARD DUCK

So MallardDuck's quack is a real live duck **quack**, not a **squeak** and not a **mute quack**.

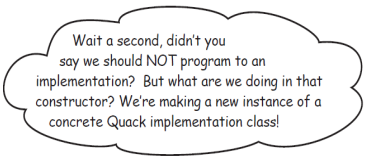
When a MallardDuck is instantiated:

- Its constructor initializes the MallardDuck's inherited quackBehaviour instance variable to a **new instance** of type Quack (an implementation of the QuackBehaviour interface).

The same is true for the duck's flying behavior:

- The MallardDuck's constructor initializes the flyBehavior instance variable with an instance of type FlyWithWings (an implementation of the FlyBehavior interface).

CONTRADICTION...



Later in this course we'll have more patterns in our toolbox that can help us fix it.

While we *are* setting the behaviors to concrete classes, we could *easily* change that at runtime.

We still have a lot of flexibility here, but we're doing a poor job of initializing the instance variables in a flexible way.

Since the quackBehavior instance variable is an interface type, we could (through the magic of polymorphism) dynamically assign a different QuackBehavior implementation class at runtime.

OUR IMPROVED DUCK CLASS

```
public abstract class Duck{

  protected QuackBehaviour quackBehaviour;
  protected FlyBehaviour flyBehaviour;

  public Duck(){

  }

  public abstract void display();

  public void performQuack(){
    quackBehaviour.quack();
  }

  public void performFly(){
    flyBehaviour.fly();
  }

  public void swim() {
    System.out.println("swim swim swim.");
  }
}
```

Declare two reference variables for the behavior interface types. All duck subclasses (in the same package) inherit these.

Delegate to the behavior class.

THE BEHAVIOUR INTERFACES

```
public interface FlyBehaviour{
    public void fly();
}
```

The interface that all flying behavior classes implement.

```
public class FlyWithWings implements FlyBehaviour{
    public void fly() {
        System.out.println("Flies gracefully");
    }
}
```

Flying behavior implementation for ducks that DO fly...

```
public class FlyNoWay implements FlyBehaviour{
    public void fly() {
        System.out.println("I can't fly...");
    }
}
```

Flying behavior implementation for ducks that do NOT fly (like rubber ducks and decoy ducks).

This same approach is applied to the QuackBehaviour and Quack implementations

AND NOW THE TEST PROGRAM...

```
public class Main{

    public static void main(String[] args){
        Duck mallard = new MallardDuck();
        mallard.performQuack();
        mallard.performFly();
    }
}
```

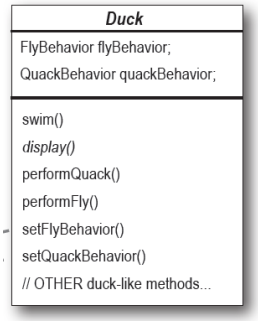
This calls the MallardDuck's inherited performQuack() method, which then delegates to the object's QuackBehavior (i.e., calls quack() on the duck's inherited quackBehavior reference).

Then we do the same thing with MallardDuck's inherited performFly() method.

```
% java Main
Quack Quack
Flies gracefully
```

SETTING BEHAVIOUR DYNAMICALLY

Imagine you want to set the duck's behavior type through a setter method on the duck subclass, rather than by instantiating it in the duck's constructor.



TWO NEW METHODS FOR THE DUCK CLASS

```
public void setFlyBehaviour(FlyBehaviour fb) {
  flyBehaviour = fb;
}

public void setQuackBehavior(QuackBehaviour qb) {
  quackBehaviour = qb;
}
```

We can call these methods anytime we want to change the behavior of a duck **on the fly**.

A NEW DUCK: MODELDUCK

```
public class ModelDuck extends Duck {

  public ModelDuck(){
    flyBehaviour = new FlyNoWay();
    quackBehaviour = new Quack();
  }

  @Override
  public void display() {
    System.out.println("I am a model Duck");
  }
}
```

Our model duck begins life grounded... without a way to fly.

ADDING A NEW FLYBEHAVIOUR

That's okay, we're creating a rocket-powered flying behavior.

```
public class FlyRocketPowered implements FlyBehaviour{

  public void fly() {
    System.out.println("I'm flying using a rocket!");
  }
}
```

CHANGING THE TEST CLASS

```
public class Main{

    public static void main(String[] args){

        Duck model = new ModelDuck();
        model.performQuack();
        model.performFly();
        model.setFlyBehaviour(new FlyRocketPowered());
        model.performFly();
    }
}
```

The first call to performFly delegates to the FlyBehaviour object set by the ModelDuck's constructor.

This invokes the model's inherited behaviour setter method. The ModelDuck's flying behaviour is now different!

The ModelDuck dynamically changed its flying behaviour.

```
% java Main
Quack Quack
I can't fly...
I'm flying using a rocket!
```

THE BIG PICTURE

We've started to describe things a little differently.

Instead of thinking of the duck behaviors as a **set of behaviors**, we'll start thinking of them as a **family of algorithms**.

- For our Duck sim, the algorithms represent the things a duck would do.

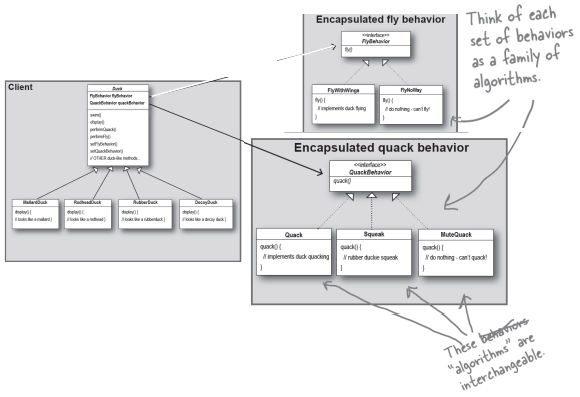
In another system the algorithms might be:

- Computing tax for different types of organization.
- Using different types of sort (java.util.Comparator).
- Different encryption algorithms.

YOU JUST USED THE STRATEGY PATTERN

YAY!

DUCKS IN SUMMARY



WHAT WE HAVE LEARNT?

HAS-A can be better than **IS-A:**

- The HAS-A relationship is an interesting one: each duck has a FlyBehavior and a QuackBehavior to which it delegates flying and quacking.

What we have done is use **composition**.

- Instead of *inheriting* their behavior, the ducks get their behavior by being *composed* with the right behaviour object.

This is an important technique; in fact, we've been using our third design principle:



Design Principle

Favor composition over inheritance.

CONCLUSION

Creating systems using composition gives you a lot more flexibility.

- Allows you to encapsulate a family of algorithms into their own set of classes.
- Lets you **change behaviour at runtime** as long as the object you're composing with implements the correct behaviour interface.

Composition, delegation and programming to interfaces are all design principles used in many design patterns.

- You will see them pop up throughout the module.

MIDTERM EXAM

Will be busy / lots to do.

Bring pen a pencil to draw diagrams, maybe a ruler as well.

- Don't aim for beautiful pictures.

What will the questions be?

- Basically weeks 1 – 4.
- Object Orientation.
- Behavioural Diagrams (that we have covered).
- Class diagrams to code.

LAB 5

Ducks and more ducks.

You need to take the initial DuckUSim system as described at the start of the lecture.

- Using inheritance to implement features.

And transform it into final system we discussed.

- Using composition, delegation and interfaces.

Lots of the necessary code is in these slides!